Assignment 2Report - Hamdan ul aziz

1.

QuickBite is a small food-ordering service. Customers place an order, the kitchen marks it ready, or it gets cancelled. In Assignment 1 I added logs and metrics. In this assignment I made the HTTP API better, added a second way to place an order (gRPC), and added more metrics to compare the two.

Part	Tool or port	What it will do and does
HTTP API	Node + Express, port 3000	Orders over HTTP (POST /orders, GET /orders, GET and PATCH /orders/{id})
gRPC service	Node + grpc-js, port 50051 (metrics/health 9464)	Place an order over gRPC. Runs in its own container
Postgres	port 5432	The one database both services use (single source of truth)
Swagger UI	port 8081	Shows openapi.yaml and lets me try the API
Prometheus / Grafana	ports 9090 / 3001	Collect and draw metrics
Filebeat, Elasticsearch, Kibana	ports 9200 / 5601	Collect and search logs
Python scripts	clients/ and tests/	Caller with retries, gRPC client, demos, tests

The task I chose is "place an order". The HTTP version is POST /orders and the gRPC version is OrderService.PlaceOrder. Both call the same business code in shared/orderService.js and use the same Postgres tables.

Everything runs with Docker and one command starts all containers. After that I check that they are running and that both services answer.

So i structured this report in a way where it goes from start to end in chronological order, it has the screenshot needed, the brief explanations and then i also added the commands used to generate that sepcific output if you want to test it on your end.

Screenshot 1: All containers running

I started all the containers with docker compose, then checked that the HTTP app and the gRPC service both answer on their health URLs.

```
docker compose up --build -d
docker compose ps
curl localhost:3000/health
curl localhost:9464/health
```

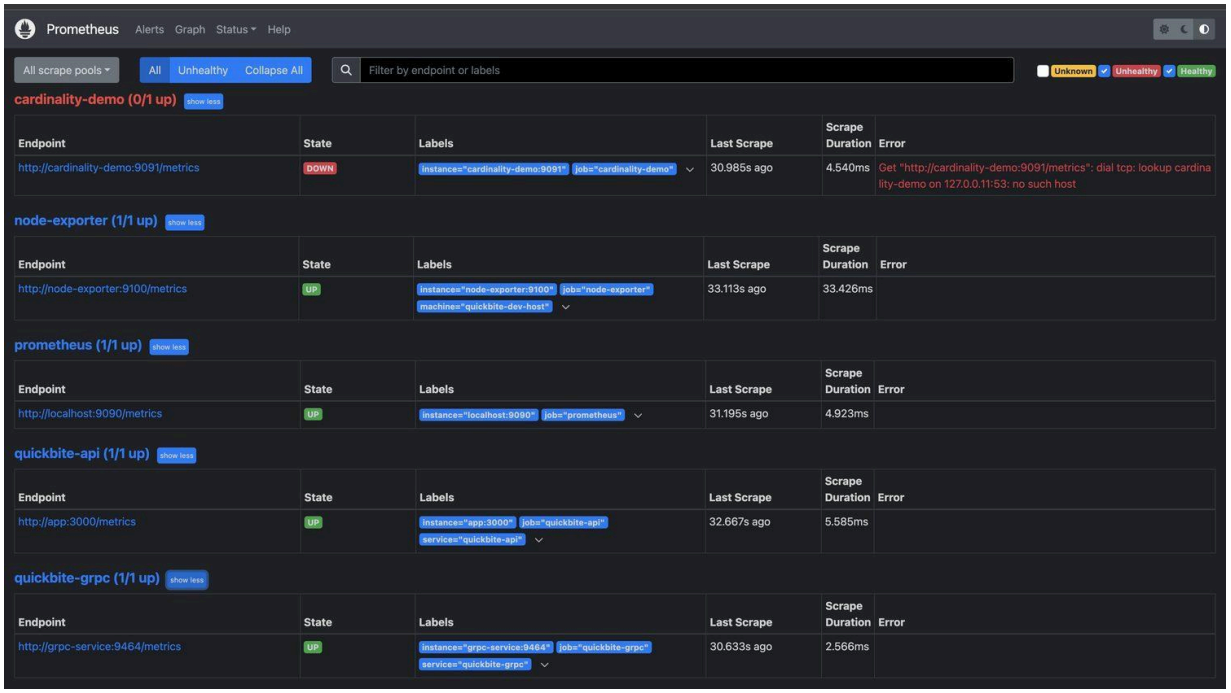
```
macenergy@Interlinks-MacBook-Pro quickbite-observability % docker compose ps
NAME                      IMAGE                                                COMMAND                  SERVICE         CREATED
 STATUS                    PORTS
quickbite-app             quickbite-observability-app                          "docker-entrypoint.s…"   app             15 seconds ago
 Up 10 seconds             0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
quickbite-elasticsearch   docker.elastic.co/elasticsearch/elasticsearch:8.14.3   "/bin/tini -- /usr/l…"   elasticsearch   15 seconds ago
 Up 13 seconds             0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
quickbite-filebeat        docker.elastic.co/beats/filebeat:8.14.3              "/usr/bin/tini -- /u…"   filebeat        15 seconds ago
 Up 10 seconds
quickbite-grafana         grafana/grafana:11.1.0                               "/run.sh"                grafana         15 seconds ago
 Up 9 seconds              0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
quickbite-grpc            quickbite-observability-grpc-service                 "docker-entrypoint.s…"   grpc-service    15 seconds ago
 Up 10 seconds             0.0.0.0:9464->9464/tcp, [::]:9464->9464/tcp, 0.0.0.0:50051->50051/tcp, [::]:50051->50051/tcp
quickbite-kibana          docker.elastic.co/kibana/kibana:8.14.3               "/bin/tini -- /usr/l…"   kibana          15 seconds ago
 Up 13 seconds             0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
quickbite-node-exporter   prom/node-exporter:v1.8.1                            "/bin/node_exporter …"   node-exporter   15 seconds ago
 Up 13 seconds             0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp
quickbite-postgres        postgres:16-alpine                                   "docker-entrypoint.s…"   postgres        2 minutes ago
 Up 14 seconds (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
quickbite-prometheus      prom/prometheus:v2.53.0                              "/bin/prometheus --c…"   prometheus      15 seconds ago
 Up 9 seconds              0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
quickbite-swagger-ui      swaggerapi/swagger-ui:v5.17.14                       "/docker-entrypoint.…"   swagger-ui      2 minutes ago
 Up 14 seconds             0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
macenergy@Interlinks-MacBook-Pro quickbite-observability % curl localhost:3000/health
{"status":"ok"}%
macenergy@Interlinks-MacBook-Pro quickbite-observability % curl localhost:9464/health
{"status":"ok"}%
macenergy@Interlinks-MacBook-Pro quickbite-observability %
```

Every container is Up and Postgres is healthy. Both health checks return okso both services are running.

Screenshot 2: Prometheus targets page

I opened the Prometheus targets page to check that it scrapes the new gRPC service.

```
open http://localhost:9090/targets
```



quickbite-api and quickbite-grpc are both UP and cardinality-demo is down because it is only for the Assinfment 1 experiment and I did not start it.

3. Part A: Improve the HTTP API

A1. The six REST constraints

Constraint	meaning	in my API
Client-server	Caller and server are separate. They only meet at the API.	curl, Swagger UI and http_caller.py call the API. None of them know about Express or Postgres.
Stateless	Each request has everything the server needs. No session is kept.	POST /orders carries the full order and the Idempotency-Key. Orders live in the database, not in a session.
Cacheable	Responses can say if they can be reused.	Partly. GET responses have an ETag. I did not set Cache-Control, and POST is not cacheable.
Uniform interface	Same simple rules everywhere: URLs for things, standard methods and codes.	Paths are nouns (/orders, /orders/{id}). Methods are GET, POST, PATCH. One error format. 201 includes a Location header. No hypermedia links.
Layered system	Client cannot tell if it talks to the real server or something in between.	Partly. Clients only see the HTTP API, never the database. I did not deploy a proxy or gateway.
Code on demand (optional)	Server may send code to run in the client.	Not used.

A2. Clean up the endpoints

I checked every app endpoint, not health or metrics, but all of them now have clear paths, the right method, input checks, meaningful status codes and one error format. The input checks are in one file (shared/validation.js) that HTTP and gRPC both use.

Endpoint	Method	Input checks	Status codes
/orders	POST	items 1 to 50; name not empty; price > 0; qty whole number 1 to 99; customerRef simple text; Idempotency-Key max 128 chars; JSON must be valid	201 (with Location), 400, 413, 422, 503
/orders	GET	status must be placed, ready or cancelled; limit 1 to 100; offset 0 or more	200, 400, 429 (with Retry-After), 503
/orders/{id}	GET	id must look like an order id	200, 404, 503
/orders/{id}	PATCH	status must be ready or cancelled	200, 400, 404, 409, 503

One error format for every error:

```
{ "error": { "code": "VALIDATION_ERROR",
             "message": "request validation failed",
             "details": [ { "field": "items[0].qty", "issue": "must be an integer between 1 and
99" } ],
             "requestId": "ddf71594-..." } }
```

A3. Three specific practices

1) OpenAPI

OpenAPI is a standard file format (YAML or JSON) that describes an HTTP API, its paths, methods, inputs, success answers and error answers. It is used because tools can read it to make documentation like Swagger UI, tests and client code, and everyone agrees on one description of the API.

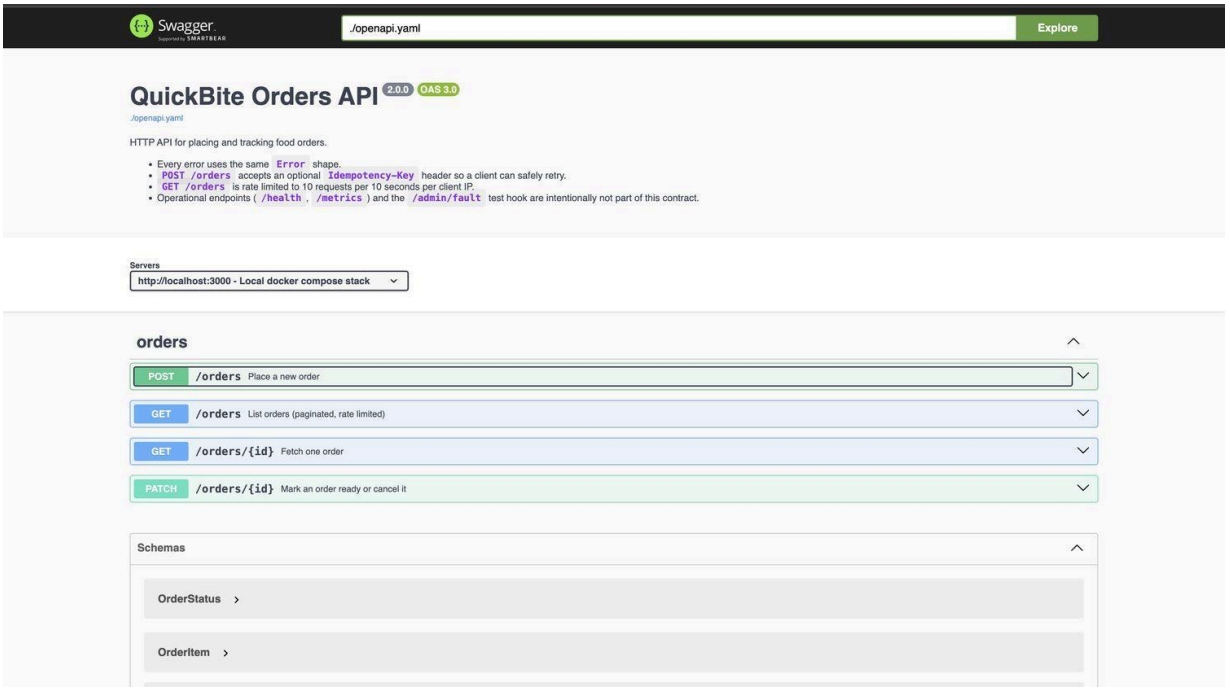
openapi.yaml (OpenAPI 3.0.3) is my file and it covers all four app endpoints, every input, body, query, Idempotency-Key, X-Request-ID and all success and error responses, including 429 with Retry-After. Health, metrics and the admin fault hook are left out on purpose. The file is checked by openapi-spec-validator in the tests.

I opened it in Swagger UI and tried POST /orders which are screenshots 3 to 6 and the automated tests also check every real response against this file.

Screenshot 3: Swagger UI showing openapi.yaml

I opened Swagger UI, which loads my openapi.yaml.

```
open http://localhost:8081
```

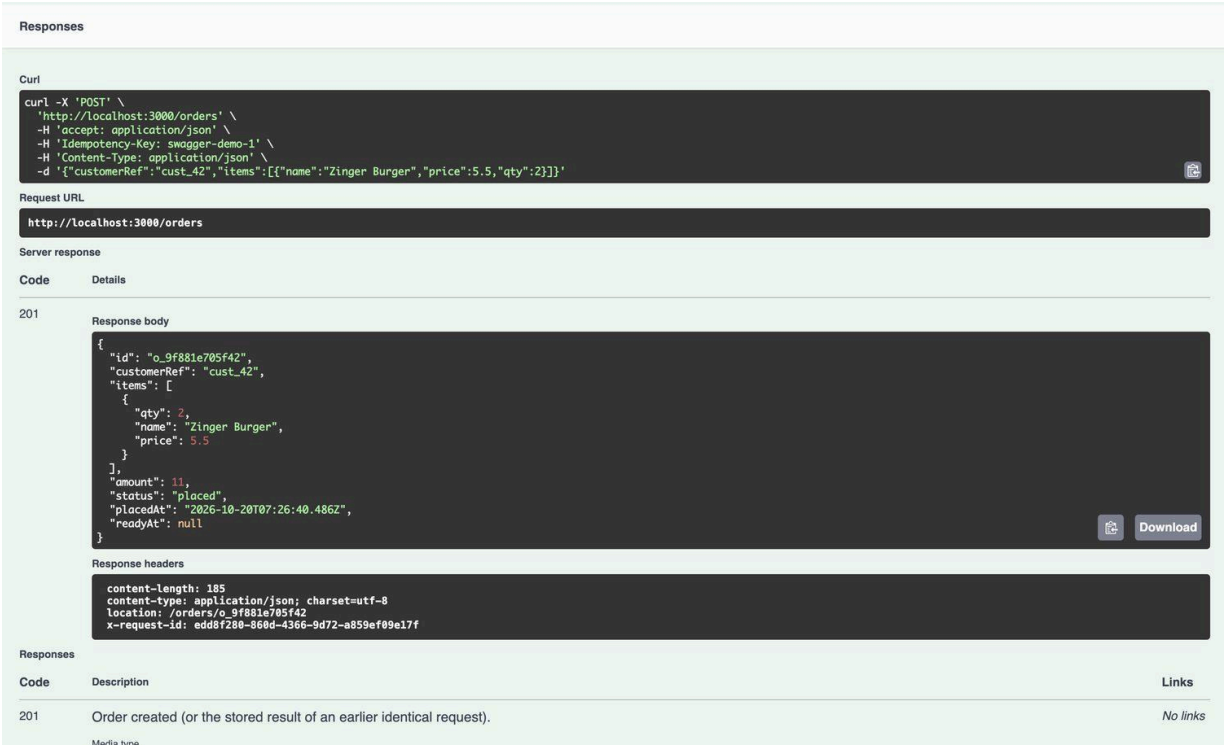


It lists all four endpoints and the schemas, so the spec matches my routes.

Screenshot 4: Swagger "Try it out": POST /orders succeeds

In Swagger I tried POST /orders with Idempotency-Key swagger-demo-1 and an order of 2 Zinger Burgers.

```
Swagger sends:
curl -X 'POST' 'http://localhost:3000/orders' -H 'Idempotency-Key: swagger-demo-1' -H
'Content-Type: application/json' \
  -d '{"customerRef":"cust_42","items":[{"name":"Zinger Burger","price":5.5,"qty":2}]}'
```

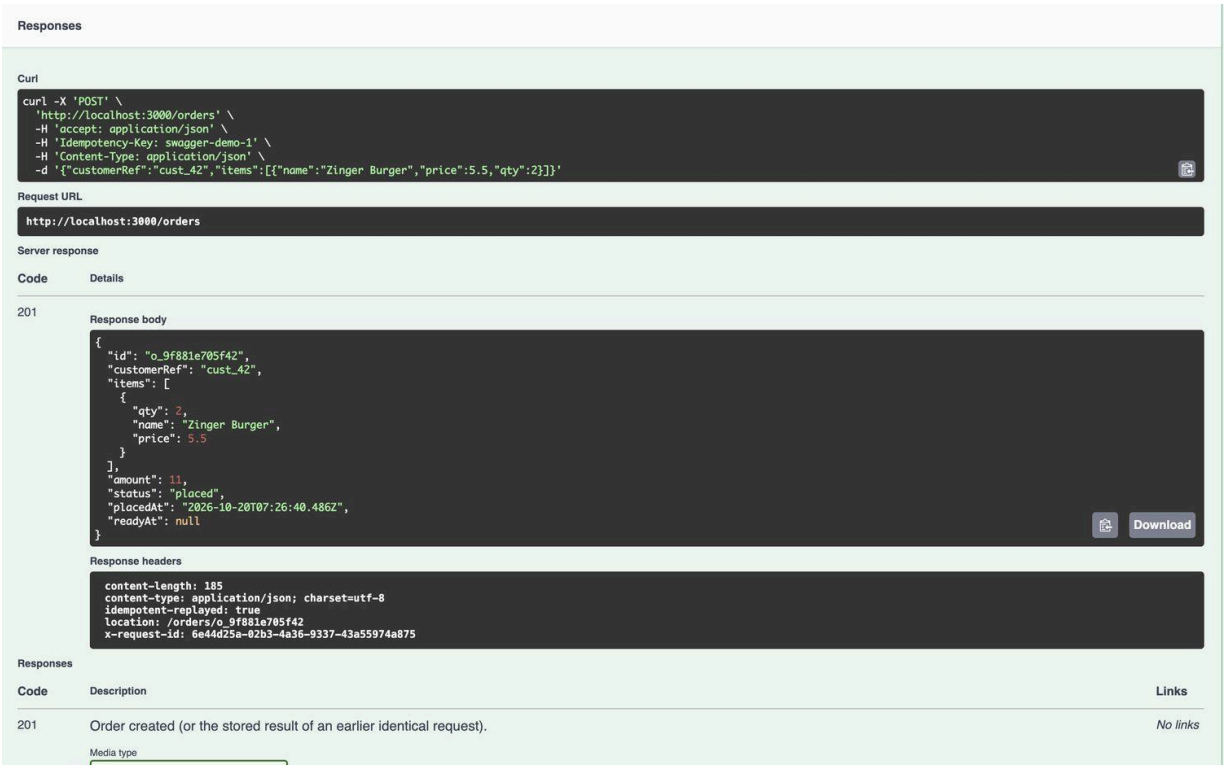


I got 201 with the new order and a Location header, which is the success response in the spec.

Screenshot 5: Swagger: same request again (idempotency)

I clicked Execute again with the same key and the same body.

```
(same curl as Screenshot 4, same Idempotency-Key: swagger-demo-1)
```

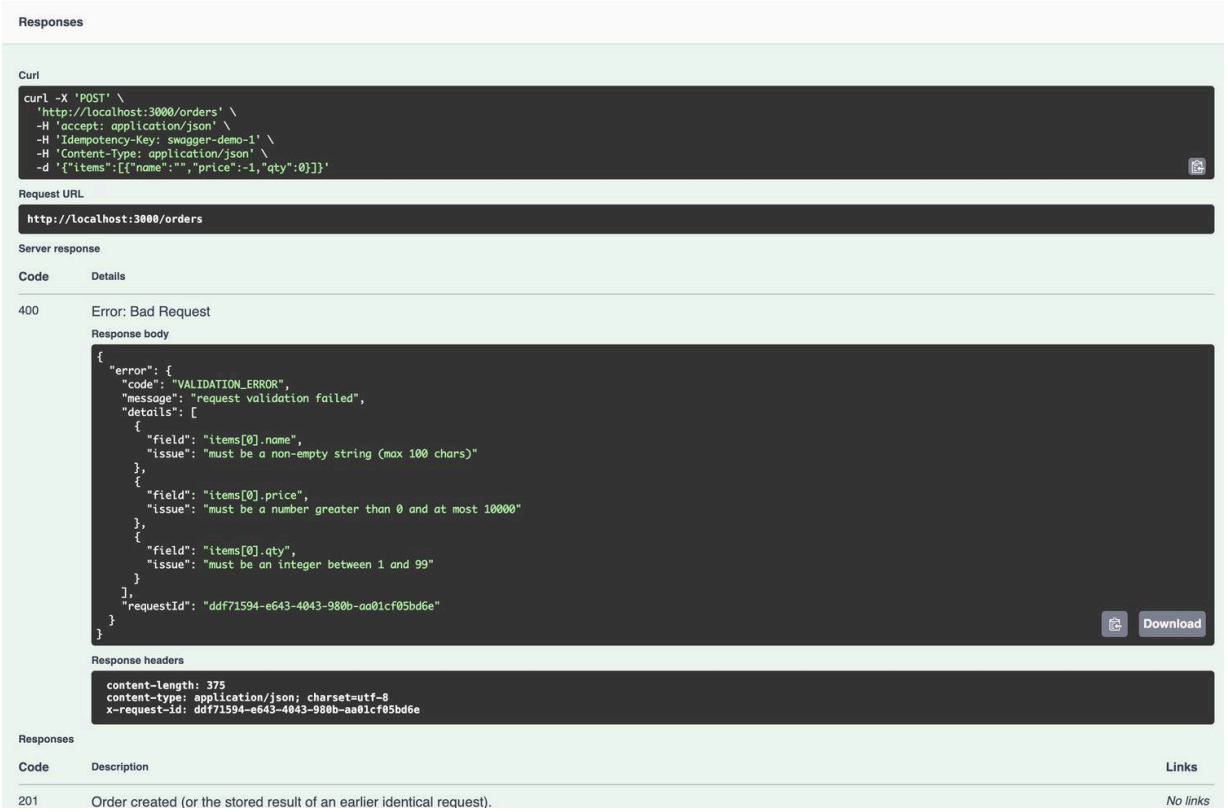


The same order id came back with idempotent-replayed: true, so no second order was made.

Screenshot 6: Swagger: invalid input gives 400

I sent a bad order with an empty name, price -1 and qty 0.

```
curl -X 'POST' 'http://localhost:3000/orders' -H 'Content-Type: application/json' -d
'{"items":[{"name":"","price":-1,"qty":0}]}'
```



I got 400 VALIDATION_ERROR with all three fields listed and a requestId, in the standard error format.

2) Idempotency

Operation: POST /orders. A caller may retry it after a timeout and must not get two orders.

the caller sends an Idempotency-Key header and in one database transaction the service saves the key (it is a primary key), then the order. If the same key comes again with the same body, the saved order is returned and the header Idempotent-Replayed: true is added. If the same key comes with a different body, the answer is 422. If two repeats arrive at the same time, the primary key makes the second one wait and then replay.

PATCH /orders/{id} is already safe to repeat because it sets a target status. Marking an order "ready" twice gives 200 both times.

Screenshot 7: curl: place an order twice with the same key

I sent the same POST /orders twice with the same Idempotency-Key.

```
curl -i -X POST localhost:3000/orders -H 'Content-Type: application/json' -H 'Idempotency-Key:
demo-1' -H 'X-Request-ID: demo-http-1' \
  -d '{"customerRef":"cust_42","items":[{"name":"Zinger Burger","price":5.5,"qty":2}]}'
(run the same command a second time)
```



Both give 201 with the same order id, and the second one has Idempotent-Replayed: true.

Screenshot 8: curl: a body that is not valid JSON

I sent a body that was not valid JSON to see how the API handles it.

```
curl -i -X POST localhost:3000/orders -H 'Content-Type: application/json' -H 'X-Request-ID: demo-http-2' -d '<broken JSON body>'
```



It returns 400 INVALID_JSON in the same error format and does not crash.

3) Rate limiting

Item	Value
Limited endpoint	GET /orders (the list, the most expensive read). Code: app/rateLimit.js
Limit	10 requests per 10 seconds (fixed window)
Who shares it	Everyone with the same client IP shares one counter for this endpoint. Other endpoints are not limited.
When allowed again	When the current 10-second window ends. The 429 answer says how long to wait in the Retry-After header.

Screenshot 9: Rate limit demo: 200s, then 429, then 200 again

iran the rate limit script. It calls GET /orders 13 times, waits for Retry-After then calls twice more.

```
python3 clients/rate_limit_demo.py
```



Requests 1 to 10 return 200, requests 11 to 13 return 429 with Retry-After=10s, and after the wait it works again.

A4. Before and after: "place an order"

Problem	Before (Assignment 1)	After (Assignment 2)
Endpoint	POST /orders	POST /orders (same URL, stronger rules). Callers do not need a new URL.
Validation	Only checked that items was a non-empty list. Negative prices and qty "abc" were accepted.	Full rules in shared/validation.js and all problems listed together. Bad data never reaches the database.
Retries	A retry after a timeout made a second order.	Optional Idempotency-Key. Same key + same body returns the first order. Retrying is now safe.
Errors	Different shapes like {"error":"text"}, no request id.	One Error shape with code, details and requestId. Callers can react to the code and I can find the log line.
Response	201 with body only. Money used floating point.	201 plus Location header. Money is counted in whole cents, so no rounding errors.
Storage	Orders kept in memory and lost on restart.	Postgres. This was needed so the gRPC service can use the same data.

Other endpoints:

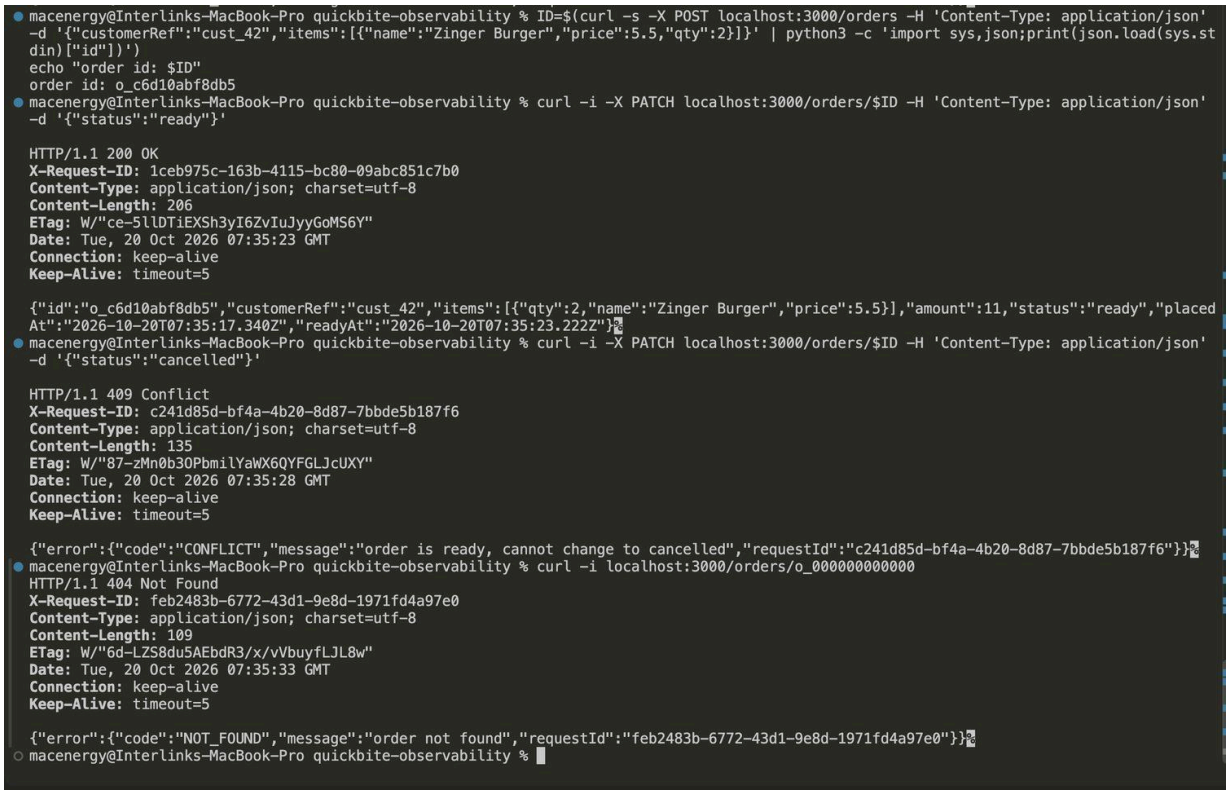
Endpoint	Result	what/why
GET /orders	Changed	Added limit, offset and status filter, result shaped as data + pagination, and the rate limit.
GET /orders/{id}	Changed	Bad ids give 404 without touching the database. Same error format.
POST /orders/{id}/ready and /cancel	Replaced	Now one endpoint: PATCH /orders/{id} with {"status":"ready"} or {"status":"cancelled"}. Repeating a status is a safe 200. Changing a finished order to a different status is 409.
POST /admin/fault	Changed	Test hook only. Input is validated and it can fail or slow the next orders for the retry demo. Not in the OpenAPI file.
GET /health, GET /metrics	Left alone	Operational endpoints, not part of this review.

The callers i updated are experiments/load_generator.js now uses PATCH and an Idempotency-Key. The README and docs use the new calls. fault_experiment.sh needed no change.

Screenshot 10: Order lifecycle with PATCH

I created an order, marked it ready, tried to cancel it, then asked for an order that does not exist.

```
ID=$(curl -s -X POST localhost:3000/orders -H 'Content-Type: application/json' -d
'{"customerRef":"cust_42","items":[{"name":"Zinger Burger","price":5.5,"qty":2}]}' | python3 -c
'import sys,json;print(json.load(sys.stdin)["id"])')
echo "order id: $ID"
curl -i -X PATCH localhost:3000/orders/$ID -H 'Content-Type: application/json' -d
'{"status":"ready"}'
curl -i -X PATCH localhost:3000/orders/$ID -H 'Content-Type: application/json' -d
'{"status":"cancelled"}'
curl -i localhost:3000/orders/o_000000000000
```



Ready gives 200, cancelling a ready order gives 409, and the unknown order gives 404. All errors use the same format.

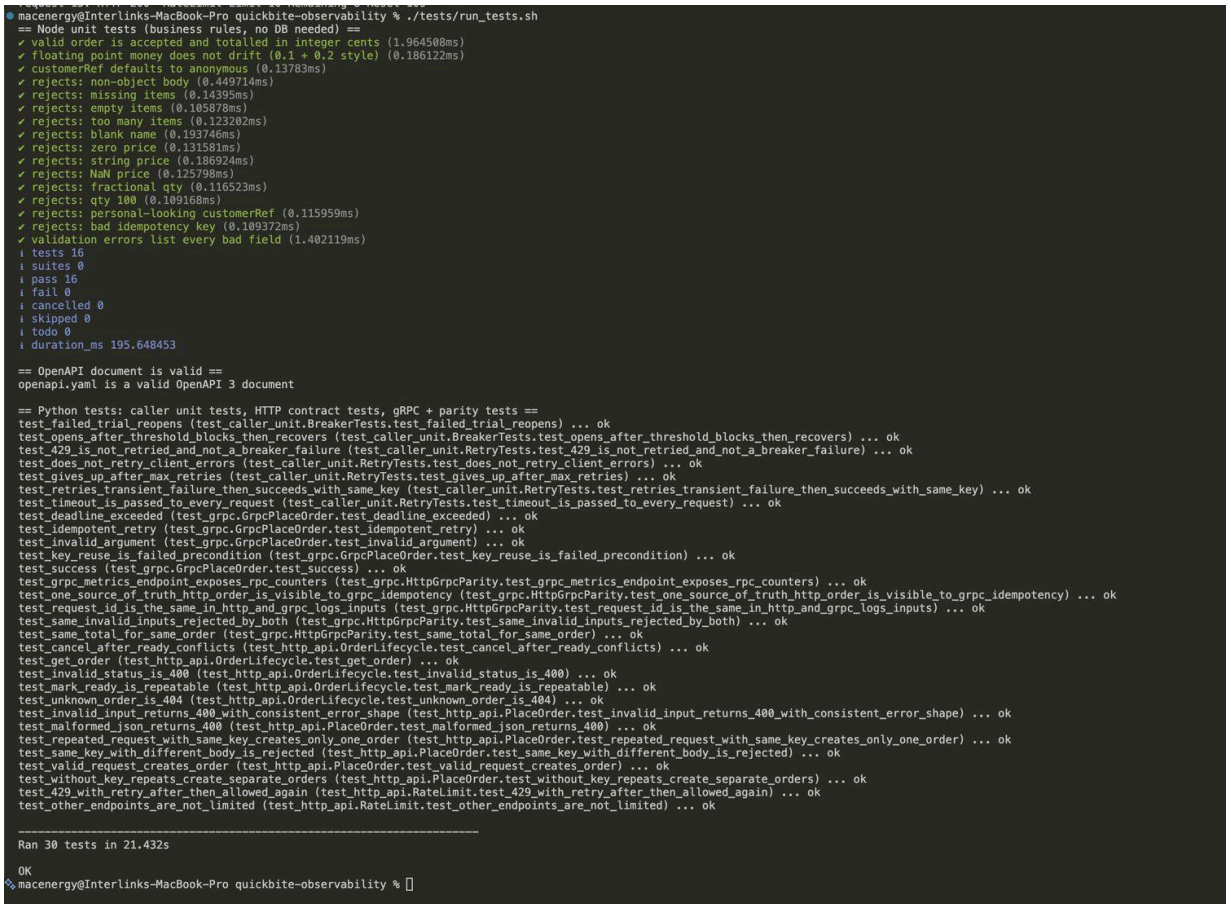
A5. Testing

One script runs everything so node unit tests for the business rules, a check that openapi.yaml is valid, and Python tests that call the running API. I tested a valid request, invalid input and a repeated request plus rate limit, 404, 409, and the gRPC checks used in Part B

Screenshot 11: All tests pass

I ran the full test script with the stack running.

```
./tests/run_tests.sh
```



The 16 Node tests pass and all 30 Python tests are OK, including the valid, invalid and repeated request tests.

A6. Python caller: timeout, retries, circuit breaker

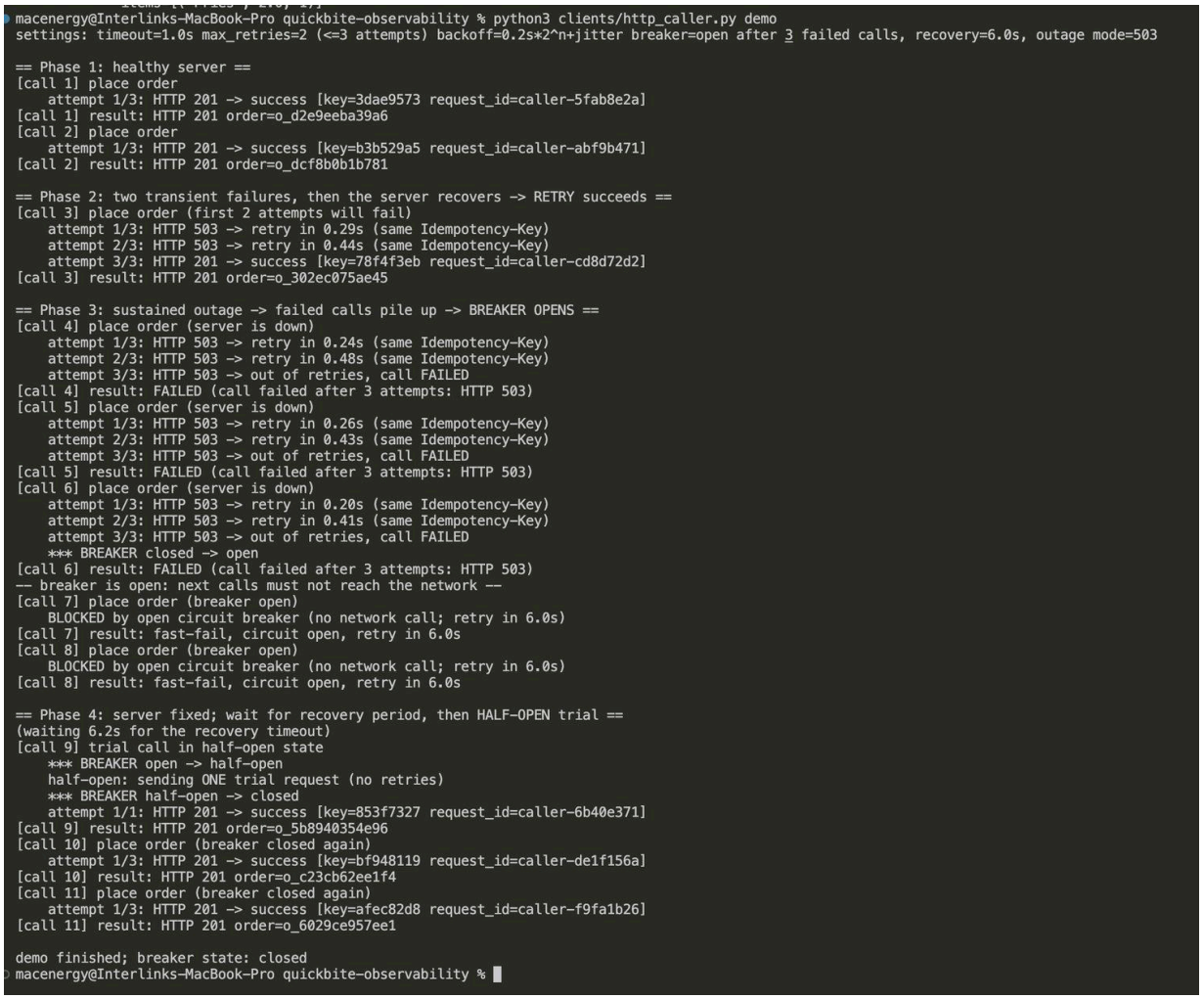
clients/http_caller.py pretends to be another service that places orders. It makes several calls per run. below are its settings

Setting	Value
Timeout	1.0 second per attempt
Retry limit	2 retries (up to 3 attempts). Wait 0.2 s x 2^n plus a little random jitter (about 0.2 s, then 0.4 s)
What is retried	Only temporary failures: timeout, connection error, 502, 503, 504. Every attempt uses the same Idempotency-Key, so a retry can never make a duplicate order. 4xx errors and 429 are not retried.
Circuit breaker	Opens after 3 failed calls in a row. Stays open for 6 seconds and blocks calls without using the network. Then it lets one trial call through (half-open). Success closes it, failure opens it again.
How failures are triggered	A server error (503) or a timeout (slow server). Not the rate limit, which is shown separately in Screenshot 9.

Screenshot 12: Caller demo with server errors (503)

I ran the caller demo where the server returns 503 on purpose.

```
python3 clients/http_caller.py demo
```

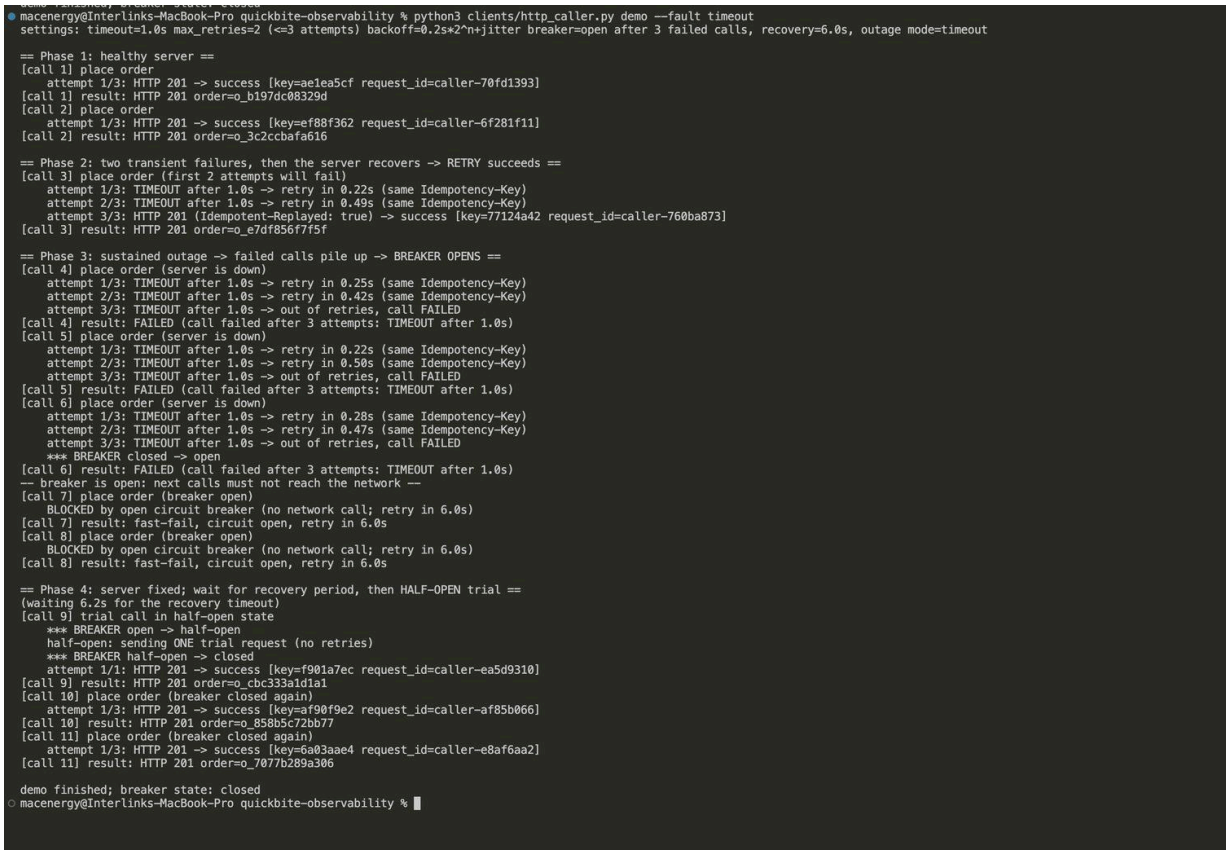


It succeeds, retries two 503s and succeeds on the third try, opens the breaker after three failed calls, blocks calls 7 and 8, then recovers.

Screenshot 13: Caller demo with timeouts

I ran the same demo but with a slow server so the client times out.

```
python3 clients/http_caller.py demo --fault timeout
```



The same 4 steps now with timeouts. On call 3 the last attempt was a replay, so the retry did not create a duplicate order.

4. Part B: gRPC service

Operation chosen: place an order.

So the HTTP version stays as it is. The gRPC version runs in a separate container (quickbite-grpc).

B1. The .proto file and generated code

proto/orders.proto defines one unary RPC and its messages:

```
service OrderService {
  rpc PlaceOrder(PlaceOrderRequest) returns (Order);
}
// messages: OrderItem, PlaceOrderRequest (customer_ref, items, idempotency_key), Order
```

Generation command (makes the Python client code):

```
./scripts/gen_proto.sh
# which runs:
python3 -m grpc_tools.protoc -I proto --python_out=clients/generated
--grpc_python_out=clients/generated proto/orders.proto
```

This creates clients/generated/orders_pb2.py (messages) and orders_pb2_grpc.py (the OrderServiceStub). The Node server (grpc-service/server.js) loads the same .proto file when it starts and I wrote the PlaceOrder handler myself.

B2. Same rules, same data

The gRPC handler does not call the HTTP API. Both servers call shared/orderService.js, which does the validation, idempotency and SQL on the same Postgres tables. The tests prove it: the same bad input is rejected by both, the same order gets the same total, and an order placed over HTTP with a key is replayed when gRPC uses that key.

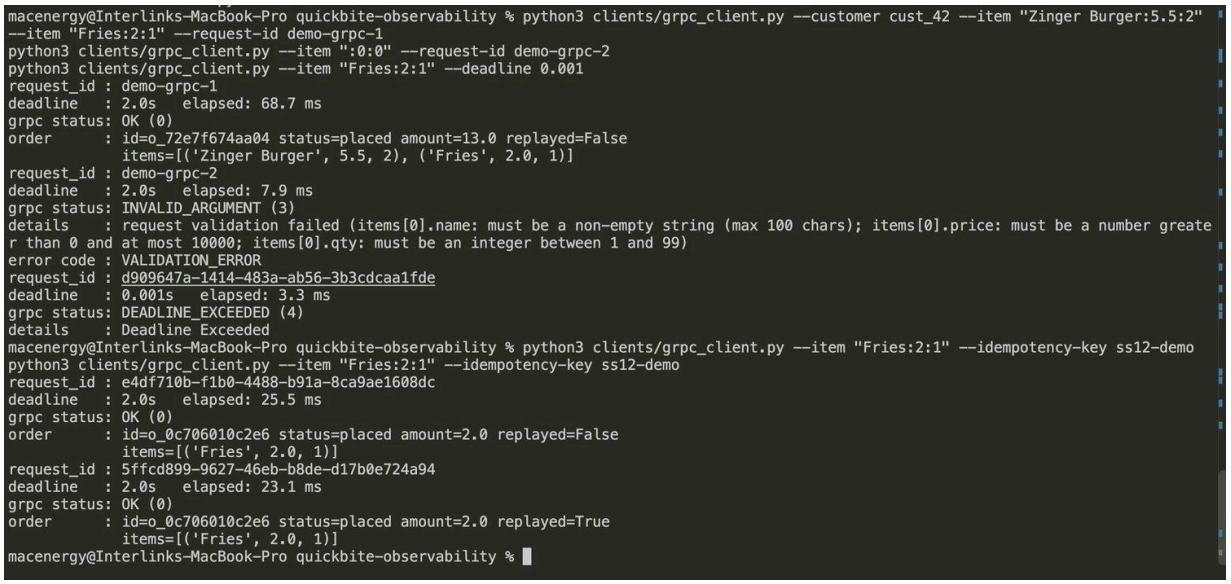
B3. gRPC client and results

clients/grpc_client.py takes the order as input, sets a deadline (default 2 seconds), sends an x-request-id, and prints the result or the error.

Screenshot 14: gRPC client: success, error, deadline, and idempotency

I ran the gRPC client for a good order, an invalid order, a 0.001 s deadline, and the same idempotency key twice.

```
python3 clients/grpc_client.py --customer cust_42 --item "Zinger Burger:5.5:2" --item "Fries:2:1"
--request-id demo-grpc-1
python3 clients/grpc_client.py --item ":0:0" --request-id demo-grpc-2
python3 clients/grpc_client.py --item "Fries:2:1" --deadline 0.001
python3 clients/grpc_client.py --item "Fries:2:1" --idempotency-key ss12-demo   (run twice)
```



I got OK, INVALID_ARGUMENT and DEADLINE_EXCEEDED. The second call with the same key returned the same order with replayed=True.

HTTP and gRPC side by side

Situation	HTTP	gRPC	Why
Success	201 Created + Location (Screenshots 4, 7)	OK (0) (Screenshot 14)	The order was created.
Invalid input	400 VALIDATION_ERROR (Screenshot 6)	INVALID_ARGUMENT (3) (Screenshot 14)	The caller sent wrong data. Retrying the same data will not help.
Same key, different body	422	FAILED_PRECONDITION (9)	Request is well formed but conflicts with the saved key.
Order not found	404	NOT_FOUND (5)	Nothing with that id.
Database down	503	UNAVAILABLE (14)	Temporary. Safe to retry with a key.
Caller gave up waiting	Client timeout (no status)	DEADLINE_EXCEEDED (4) (Screenshot 14)	The deadline passed before the answer came.

B4. Diagram of both call paths

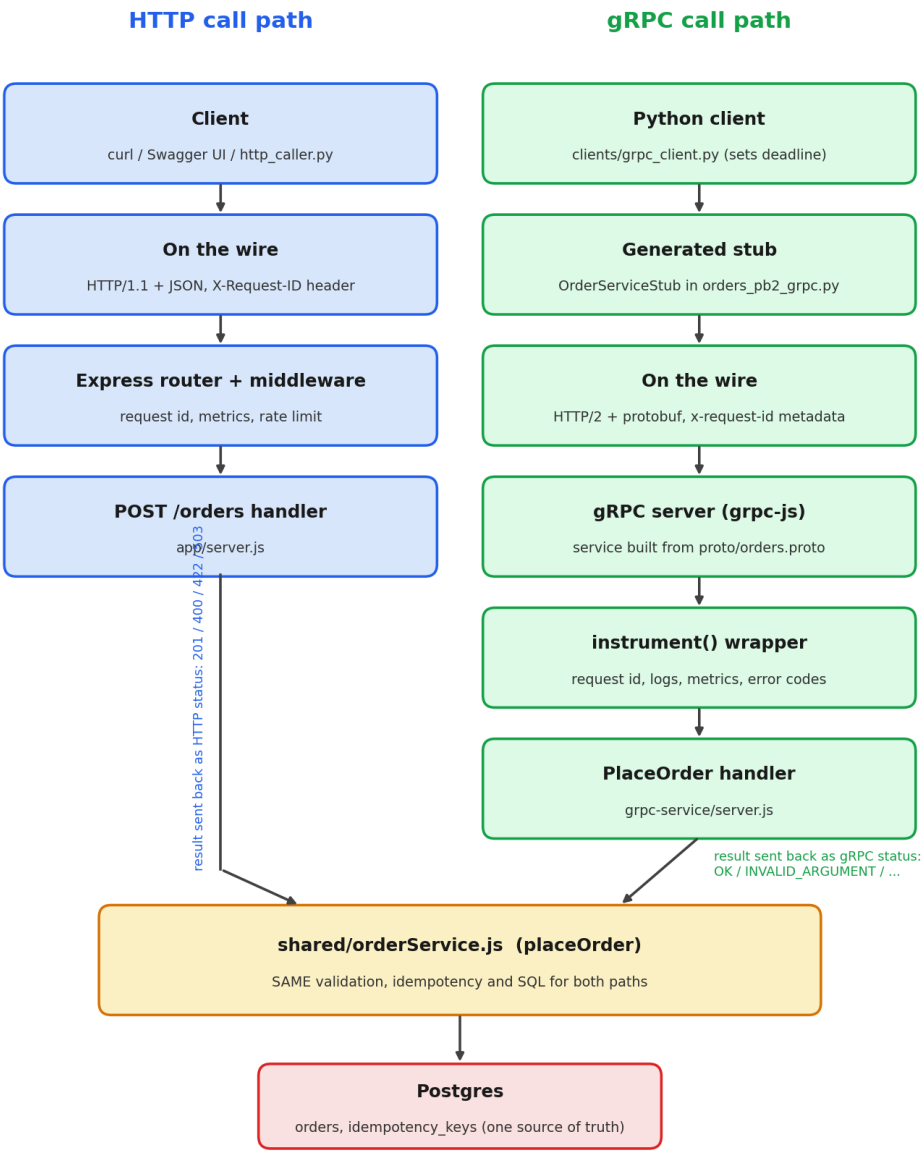


Diagram: HTTP path (left) and gRPC path (right) meet at the same shared code and database.

The gRPC side shows the generated stub and the server handler. Only the front door is different. From orderService down, both paths run the same code.

B5. Trace IDs across the gRPC boundary

From Assignment 1 every request has a requestId. HTTP reads it from the X-Request-ID header. gRPC reads it from the x-request-id metadata key. Both services write it in their JSON logs, and Filebeat sends the logs to Elasticsearch.

Screenshot 15: One request id sent to HTTP and gRPC

I sent one HTTP order and one gRPC order with the same request id, corr-shared-42.

```
curl -s -o /dev/null -X POST localhost:3000/orders -H 'Content-Type: application/json' -H
'X-Request-ID: corr-shared-42' -d '{"items":[{"name":"Fries","price":2,"qty":1}]}'
python3 clients/grpc_client.py --item "Fries:2:1" --request-id corr-shared-42
```

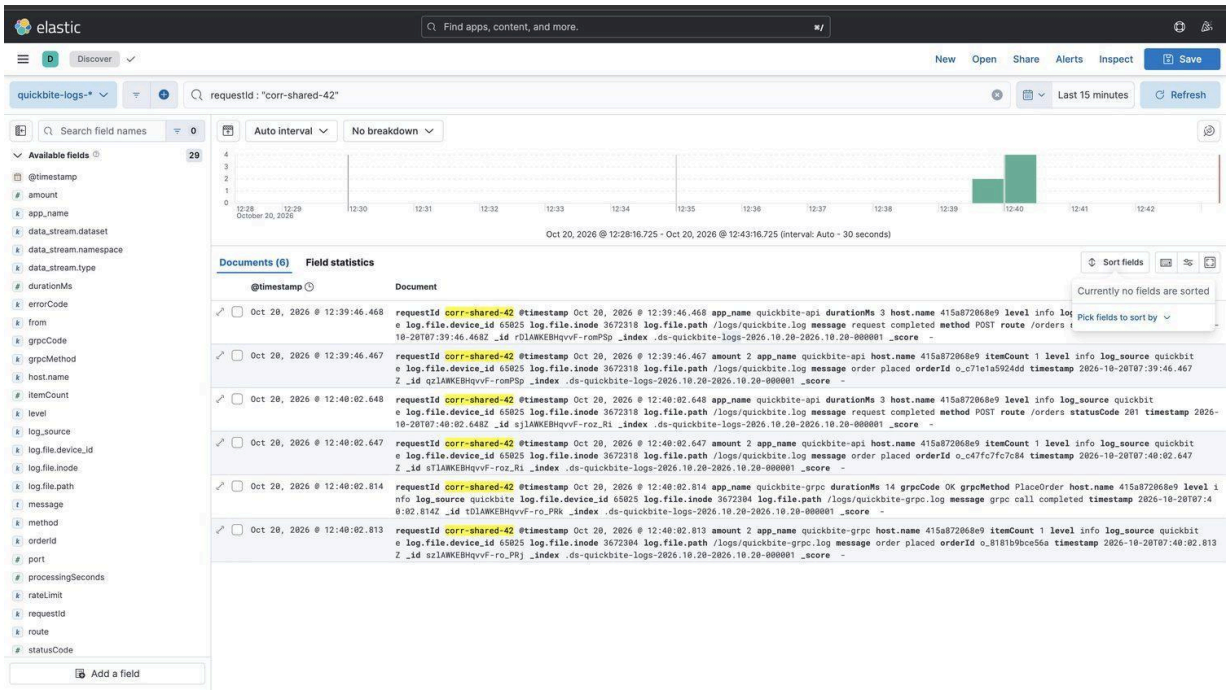
```
macenergy@Interlinks-MacBook-Pro quickbite-observability % curl -s -o /dev/null -X POST localhost:3000/orders -H 'Content-Type: applicatio
n/json' -H 'X-Request-ID: corr-shared-42' -d '{"items":[{"name":"Fries","price":2,"qty":1}]}'
python3 clients/grpc_client.py --item "Fries:2:1" --request-id corr-shared-42
request_id : corr-shared-42
deadline   : 2.0s   elapsed: 19.0 ms
grpc status: OK (0)
order      : id=o_8181b9bce56a status=placed amount=2.0 replayed=False
             items=[('Fries', 2.0, 1)]
```

The gRPC client prints the same request id and OK, so both calls carry the same id.

Screenshot 16: Kibana: both services under the same id

I searched for that request id in Kibana Discover.

```
open http://localhost:5601   (search:  requestId : "corr-shared-42")
```



Logs from both quickbite-api and quickbite-grpc show up with the same requestId, so the id carries across the gRPC call.

5. Part C: Extend the metrics

C1. Audit of my Assignment 1 metrics

A1 metric	Useful yes or no	What was missing or wrong	The correct fix then
orders_total{status}	Yes	The "ready" outcome was never counted.	Now counted in shared code, so HTTP and gRPC are both included.
orders_pending	Yes	Counted in memory. Wrong with two services or after a restart.	Read from Postgres when Prometheus scrapes.
order_processing_seconds	Yes	Nothing wrong.	Kept.
http_request_duration_seconds	Yes	Smallest bucket was 10 ms, but the API answers in a few ms. Unknown URLs used the raw path as a label.	Buckets start at 1 ms. Unknown routes are labelled "unmatched".
http_requests_total	Yes	Same raw path label problem.	Same fix.
demo_requests_total{request_id}	Experiment only	High cardinality on purpose.	Left alone, kept separate.
In-flight calls (missing)	-	No way to see how busy the service is.	Added for HTTP and gRPC.
Database time (missing)	-	A1 had no database.	Added db_query_duration_seconds.
Replays, rejections, 429s (missing)	-	New behaviour was invisible.	Added three counters.
gRPC calls, errors, duration (missing)	-	gRPC service is new.	Added grpc_server_* metrics.

C2. New and changed metrics

Prometheus scrapes the gRPC service as job quickbite-grpc at grpc-service:9464/metrics (prometheus/prometheus.yml, Screenshot 2).

Duration is measured at the server not the client. For HTTP it is from request start to the response finishing. For gRPC it is from the start of the handler wrapper until the answer is handed back. The Python scripts only print their own elapsed time and have no metrics endpoint.

Labels are bounded as I never use user, request, order or trace ids as labels. Label values come from small fixed sets (method name, gRPC code, route, operation name, reason).

Metric	Purpose	Type	Unit	Labels / code location	Query
grpc_server_handled_total	gRPC calls and error rate	counter	calls	grpc_method, grpc_code grpc-service/server.js	sum by (grpc_method, grpc_code) (rate(grpc_server_handled_total[1m]))
grpc_server_handling_seconds	gRPC duration (p50, p95)	histogram	seconds	grpc_method, grpc_code grpc-service/server.js	histogram_quantile(0.95, sum by (le) (rate(grpc_server_handling_seconds_bucket{grpc_method="PlaceOrder"}[1m]))) (use 0.5 for p50)

Metric	Purpose	Type	Unit	Labels / code location	Query
grpc_server_in_flight_requests	Calls being handled now	gauge	calls	grpc_method grpc-service/server.js	sum(grpc_server_in_flight_requests{grpc_method="PlaceOrder"})
http_requests_in_flight	Same for HTTP	gauge	requests	none app/metrics.js	sum(http_requests_in_flight)
http_request_duration_seconds (new buckets)	HTTP duration to compare	histogram	seconds	method, route, status_code app/metrics.js	histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{route="/orders",method="POST"}[1m])))
db_query_duration_seconds	Time spent in Postgres	histogram	seconds	operation (about 13 fixed names) shared/metrics.js, shared/db.js	histogram_quantile(0.95, sum by (le, operation, job) (rate(db_query_duration_seconds_bucket[1m])))
idempotent_replays_total	Retries answered from saved result	counter	replays	none shared/metrics.js, shared/orderService.js	sum(rate(idempotent_replays_total[5m])) * 60
orders_rejected_total	Orders refused by rules	counter	orders	reason: validation or idempotency_conflict shared/orderService.js	sum by (reason) (rate(orders_rejected_total[5m])) * 60
http_rate_limited_total	Requests refused with 429	counter	requests	route app/metrics.js, app/server.js	sum by (route) (rate(http_rate_limited_total[5m])) * 60
orders_pending (changed)	Queue size, now correct	gauge	orders	none shared/metrics.js	orders_pending
orders_total (changed)	Outcomes incl. ready	counter	orders	status shared/metrics.js	sum by (status) (rate(orders_total[5m])) * 60

Time window: rates and p50/p95 use a 1-minute window ([1m]). That is more than 4 times the 5-second scrape interval, and it reacts quickly in short tests.

Buckets: 1, 2.5, 5, 10, 25, 50, 100, 250, 500 ms, then 1 s and 2.5 s. Normal calls take a few ms and the test fault adds 300 ms, so these buckets show both the normal case and the slow tail.

Why in-flight instead of message size: PlaceOrder messages are small and limited by validation (at most 50 items), so their size hardly changes and tells little. In-flight calls show how busy the service is. When I added the 300 ms delay, in-flight calls went up while the error rate stayed normal.

Other missing metric: db_query_duration_seconds, which tells "database slow" apart from "service slow". I also added the replay, rejection and rate-limit counters.

C3. Make repeatable traffic and look at Grafana

experiments/a2_traffic.py sends the same mix every run (a fixed random seed): 70% valid orders, 20% invalid, 10% repeats of an earlier key. It sends the same load to HTTP and gRPC: 30 s normal, 30 s with +300 ms delay, 30 s recovery.

Screenshot 17: Repeatable traffic for the dashboard

I ran the traffic script and it sends 10 requests a second to HTTP and to gRPC in three phases.

```
python3 experiments/a2_traffic.py
```



It sent 707 successes and 173 errors to each protocol, so the comparison on the dashboard is fair.

Screenshot 18: Grafana: PlaceOrder over HTTP vs gRPC

I opened the Grafana dashboard and went to the Assignment 2 row.

```
open http://localhost:3001
```

```
macenergy@Interlinks-MacBook-Pro quickbite-observability % python3 experiments/a2_traffic.py
  == baseline: 30s at 10 requests/s per protocol ==  (started 12:48:55)
  == fault (+300 ms): 30s at 10 requests/s per protocol ==  (started 12:49:25)
  == recovery: 30s at 10 requests/s per protocol ==  (started 12:49:55)

  sent (status counts):
    grpc INVALID_ARGUMENT        173
    grpc OK                      707
    http 201                     707
    http 400                     173
macenergy@Interlinks-MacBook-Pro quickbite-observability %
```

HTTP and gRPC panels sit side by side, the calls, error rate, p50/p95, in-flight, DB time and all show the traffic run. The error rate is around give or take say 20%, matching the invalid traffic.

What the panels are showing here :

- Calls and errors: HTTP 201 and 400 line up with gRPC OK and INVALID_ARGUMENT and the error rate is about 20% which matches the 20% invalid traffic.
- Latency: p50 and p95 for HTTP and gRPC rise to about 0.45 s while the +300 ms delay is on and then fall back during recovery.
- In-flight: goes up during the busy period so it shows load that the rate line alone does not

- Extra panels: DB query p95, share of calls under 100 ms and business events per minute

Screenshot 19: Rest of the dashboard (Assignment 1 panels)

I scrolled up to check that the Assignment 1 panels still work so as below they do work

```
(same Grafana page, scroll up)
```

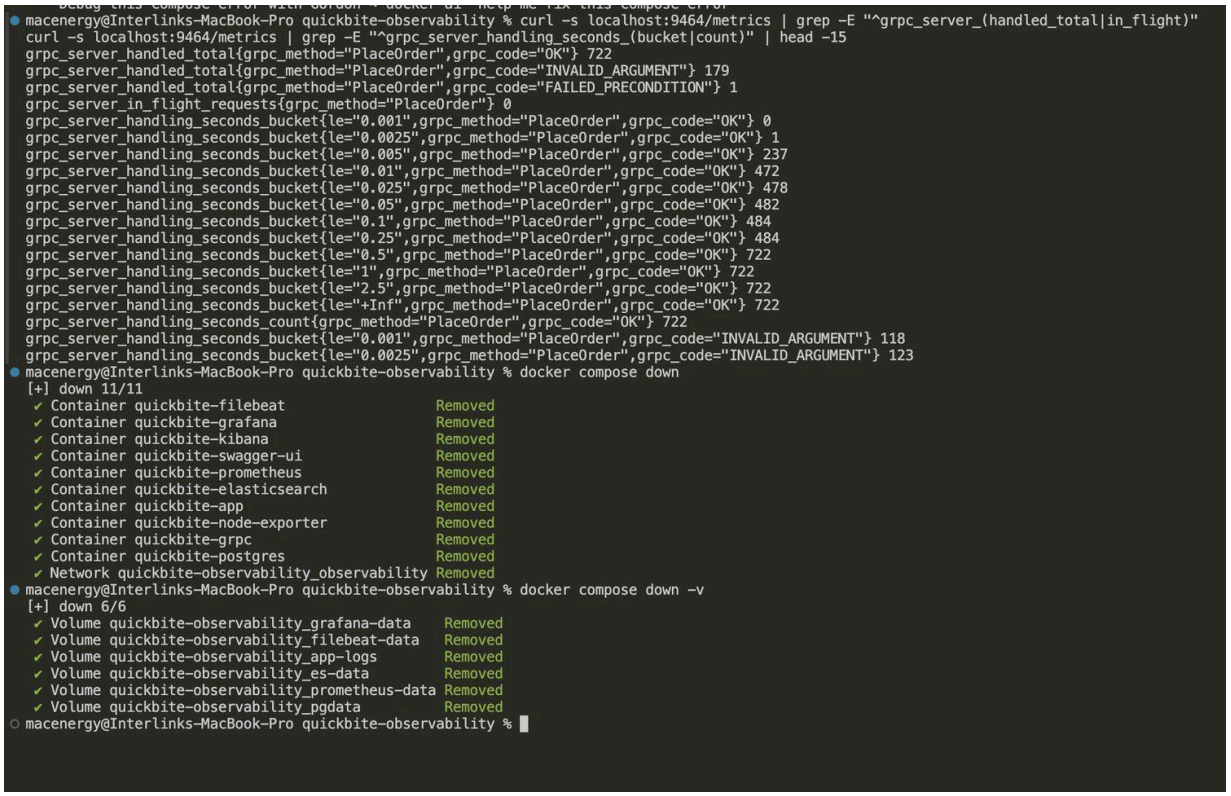


They all still show data. The cardinality panel says no data because that experiment is not running.

Screenshot 20: Raw gRPC metrics, then clean shutdown

I looked at the raw gRPC metrics then stopped everything and removed the volumes

```
curl -s localhost:9464/metrics | grep -E "^grpc_server_(handled_total|in_flight)"
curl -s localhost:9464/metrics | grep -E "^grpc_server_handling_seconds_(bucket|count)" | head
-15
docker compose down
docker compose down -v
```



The counters show OK 722, INVALID_ARGUMENT 179 and FAILED_PRECONDITION 1, plus the histogram buckets. The last two commands clean everything up as was needed in the assignment